WEEK 4 · LESSON 19 OF 24 · HANDOUT

Text as Numbers: Tokens and Embeddings

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=wjZofJX0v4M — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon Italia: Word Embeddings for Natural Language Processing in Python — https://www.youtube.com/watch?v=c8QF1FJjPG8

🎓 **Go deeper:** Harvard CS50 AI, Lecture 6: Language (attention and transformers) — https://www.youtube.com/watch?v=QAZc9xsQNjQ

Your goal in this lesson: Turn text into tokens and ids, explain word embeddings, and train a neural text classifier whose embeddings learn word meanings from the task.

Computers need numbers

A network cannot read "my mobile money pin is blocked". Text goes through three steps:

1. **Tokenise** — split the text into pieces called **tokens**: words, parts of words or characters.
2. **Index** — give every token in the **vocabulary** a number (an id).
3. **Embed** — turn every id into a short vector of learned numbers: an **embedding**.

Large language models use **sub-word tokens** (for example "unbelievable" → "un", "believ", "able"), so they can handle any word — including Kinyarwanda words they rarely saw — with a vocabulary of about 50,000–200,000 tokens. Here we use simple whole words.

The data: customer messages

A mobile network receives thousands of messages a day. We will sort them automatically into three queues — **payment**, **network** and **account** — so each reaches the right team faster. The messages are simulated from templates.

```
import re
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

TEMPLATES = {
    "payment": ["i sent money to {who} but it did not arrive", "my payment to {who} failed",
                "money was deducted twice for {who}", "how do i reverse a payment to {who}",
                "the merchant {who} did not receive my payment"],
    "network": ["my {what} is not working in {where}", "no {what} signal in {where} since
morning",
                "{what} is very slow in {where}", "i cannot connect to {what} in {where}",
                "the {what} keeps dropping in {where}"],
    "account": ["i forgot my {thing}", "please reset my {thing}", "my {thing} is blocked",
                "how do i change my {thing}", "someone asked for my {thing} on the phone"],
}
FILL = {"who": ["my brother", "the shop", "a wrong number", "the school", "my landlord", "the
supermarket"],
        "what": ["internet", "data bundle", "4g", "network", "airtime bundle"],
        "where": ["kigali", "huye", "musanze", "rubavu", "nyagatare", "my village"],
        "thing": ["pin", "password", "sim card", "mobile money pin", "account"]}
TOPICS = list(TEMPLATES)

def make_messages(n, seed):
    rng = np.random.default_rng(seed)
    texts, labels = [], []
    for _ in range(n):
        label = int(rng.integers(0, 3))
        text = re.sub(r"\{(\w+)\}", lambda m: str(rng.choice(FILL[m.group(1)])),
                      str(rng.choice(TEMPLATES[TOPICS[label]])))
        if rng.random() < 0.3:
            text = str(rng.choice(["hello ", "please ", "good morning ", "help "])) + text
        if rng.random() < 0.3:
            text += str(rng.choice([" thank you", " urgent", " please help", " today"]))
        texts.append(text)
        labels.append(label)
    return texts, torch.tensor(labels)

train_texts, y_train = make_messages(800, seed=0)
test_texts, y_test = make_messages(200, seed=1)
for t, l in zip(train_texts[:4], y_train[:4]):
    print(f"[{TOPICS[l]}] {t}")
```

Tokens and ids

```
vocab = {"<pad>": 0, "<unk>": 1}
for text in train_texts:                      # build the vocabulary from training text only
    for word in text.split():
        vocab.setdefault(word, len(vocab))

def encode(text, max_len=14):
    ids = [vocab.get(w, vocab["<unk>"]) for w in text.split()][:max_len]
    return ids + [vocab["<pad>"]] * (max_len - len(ids))

X_train = torch.tensor([encode(t) for t in train_texts])
X_test = torch.tensor([encode(t) for t in test_texts])

def fit(model, epochs=30, lr=1e-2):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X_train))
        for start in range(0, len(perm), 32):
            idx = perm[start:start + 32]
            opt.zero_grad()
            F.cross_entropy(model(X_train[idx]), y_train[idx]).backward()
            opt.step()
    model.eval()
    with torch.no_grad():
        return round((model(X_test).argmax(1) == y_test).float().mean().item(), 3)
print("vocabulary size:", len(vocab), "| first message as ids:", X_train[0].tolist())
```

`<pad>` fills short messages up to a fixed length so they fit in one tensor; `<unk>` stands for any word never seen in training.

Why not one-hot vectors?

One-hot encoding a 50,000-word vocabulary gives 50,000-long vectors where every word is equally different from every other: "internet" is as far from "4g" as from "landlord". An **embedding** is a short, dense vector (here 16 numbers; in large models thousands) that is **learned**. Words used in similar ways end up with similar vectors.

```
emb = nn.Embedding(num_embeddings=len(vocab), embedding_dim=16, padding_idx=0)
print(emb(X_train[:2]).shape)          # 2 messages x 14 tokens x 16 numbers
```

An embedding layer is just a table with one learnable row per token — looking up row 7 is the same as multiplying a one-hot vector by a weight matrix, but much faster.

A simple text classifier

Average the embeddings of the words in a message (ignoring padding), then classify:

```
class BagOfEmbeddings(nn.Module):
    def __init__(self, vocab_size, dim=16, n_classes=3):
        super().__init__()
        self.emb = nn.Embedding(vocab_size, dim, padding_idx=0)
        self.out = nn.Linear(dim, n_classes)

    def forward(self, x):
        e = self.emb(x)                                  # (batch, tokens, dim)
        mask = (x != 0).unsqueeze(-1).float()            # 1 for real words, 0 for padding
        mean = (e * mask).sum(1) / mask.sum(1).clamp(min=1)
        return self.out(mean)

torch.manual_seed(0)
bag = BagOfEmbeddings(len(vocab))
print("test accuracy:", fit(bag))
```

The model sorts the test messages almost perfectly. These template messages are much cleaner than real ones — with real customers you would expect spelling mistakes, mixed Kinyarwanda, English and French, and lower accuracy.

What did the embeddings learn?

Nobody told the model what words mean. It learned embeddings only to sort messages — yet related words moved close together:

```
E = F.normalize(bag.emb.weight.detach(), dim=1)      # unit length -> dot product = cosine
similarity
id_to_word = {i: w for w, i in vocab.items()}

def neighbours(word, k=4):
    sims = E @ E[vocab[word]]
    best = sims.argsort(descending=True)[1:k + 1]
    return [id_to_word[int(i)] for i in best]

for w in ["pin", "internet", "payment", "huye"]:
    print(f"{w:9s} ->", neighbours(w))
```

"huye" ends up near other places (kigali, nyagatare, village); "pin" near "account" and "sim". On huge text collections, the same idea (word2vec, 2013) learned famous relationships such as king − man + woman ≈ queen. Large language models learn embeddings with thousands of dimensions in exactly this spirit.

Use it

```
def route(message):
    with torch.no_grad():
        probs = F.softmax(bag(torch.tensor([encode(message)])), dim=1)[0]
    return TOPICS[int(probs.argmax())], round(float(probs.max()), 2)

print(route("my internet is not working in rubavu"))
print(route("please reset my password urgent"))
print(route("the bus was late today"))          # off-topic: watch the confidence
```

Look at the last answer: the bus message has nothing to do with the network, yet the model still picks a queue — with high confidence. A classifier can only choose among the classes it knows. Real systems add an "other" class trained on off-topic messages, or a separate check that sends unfamiliar messages to a person.

The weakness of a "bag"

Averaging ignores **word order**: "money sent to me, not by me" and "money sent by me, not to me" look identical. The next two lessons fix this with networks that read in order (RNNs) and networks that relate every word to every other (attention).

Key terms

- **Token** — a piece of text (word, sub-word or character).
- **Vocabulary** — the set of tokens the model knows, each with an id.
- **Padding** — filler ids that make sequences the same length.
- **Embedding** — a learned dense vector for each token.
- **Cosine similarity** — how closely two vectors point the same way.

Remember this: Text becomes tokens, tokens become ids, and ids become learned embeddings in which words used alike sit close together.

Name: Date:

Activity

Task 1

Change `embedding_dim` to 2 and plot every word's 2-D embedding with its label. Do topics form groups?

Task 2

Add five messages in Kinyarwanda to the training data (for example "internet ntikora i Huye"). What happens to unknown words at test time?

Task 3

Find the test messages with the lowest confidence. Why are they harder?

Task 4

Write a character-level `encode` function. What are its advantages for misspelled messages?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a token?
   - A. A password
   - B. A GPU
   - C. A label
   - D. A piece of text such as a word, sub-word or character
2. What is a word embedding?
   - A. A one-hot vector
   - B. A short learned vector of numbers for each token
   - C. The vocabulary size
   - D. The text itself
3. Why are embeddings better than one-hot vectors?
   - A. They are longer
   - B. They are short and place words used alike close together
   - C. They store the text
   - D. They need no training
4. What does the <pad> token do?
   - A. Ends the training
   - B. Marks unknown words
   - C. Fills short sequences up to a fixed length
   - D. Marks spam
5. What is the main weakness of averaging word embeddings?
   - A. It ignores word order
   - B. It needs images
   - C. It is too slow
   - D. It cannot learn
6. Why do large language models use sub-word tokens?
   - A. To avoid embeddings
   - B. To handle any word, even rare ones, with a fixed vocabulary
   - C. To make text longer
   - D. Because words are illegal

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.